

Concurrency Bugs and Synchronization

Concurrency Bugs and Synchronization

Learning Objectives:

- Be able to identify classes of concurrency bugs
- Be able to explain what mutual exclusion provides and how locks implement it

Complimentary Reading: [OSTEP](#), Locks chapter (28.1, 28.2, and 28.3) and concurrency bugs chapter (32.1 and 32.2)

Concurrency Bug Classes

- Race Condition
 - Definition: when the behavior of a program depends upon the timing or order
 - Example: Two threads call printf
 - Consequence: surprising behavior, not always a bug
- Data Race:
 - Definition: when two threads access the same memory location, at least one access is a write, and there's no mechanism that orders the accesses
 - Example: one thread performs a load from memory, the other thread writes to it
 - Consequence: undefined or surprising values (lost updates, torn reads, etc.)
- Atomicity violation
 - Definition: When correctness requires a multi-step operation to be indivisible, but thread interleavings prevent it.
 - Example: see last week's lecture
 - Consequence: bugs! Wrong results, program crashes, or others.
- Ordering violations
 - Definition: when one thread assumes that an event, X, happens before an event Y, but the program does not enforce that order.
 - Example: Assuming that initialization occurs before first use.
 - Consequence: lost events, corrupted state, etc.
- We'll add two more to this list next week!